

Personal Offline Voice-to-Text Assistant (FlowType)

This document describes a **fully offline voice transcription and typing assistant** (inspired by Wispr Flow) for Windows and Android. The assistant works by holding a hotkey to record speech, converting it to text locally, formatting it intelligently, and *automatically pasting* it at the cursor. All processing happens on-device (no cloud), using open-source libraries.

1. System Overview

- **Activation:** User holds a global hotkey (e.g. Win+Fn) to start recording and releases it to stop.
- **Recording:** Microphone audio is captured (e.g. via the `sounddevice` library) during the key press. Voice Activity Detection (VAD) ensures only speech is processed.
- **Transcription:** The audio is fed to a local **Faster-Whisper** model (quantized, CPU) to get raw text. The faster-whisper library supports multi-language (English/Hindi/Gujarati) transcription and integrates Silero VAD for silence filtering ¹.
- **Formatting:** The raw text is cleaned up. We apply simple rules (capitalization, punctuation, replacing phrases like “laughing emoji” with 😄) or, optionally, run a tiny LLM locally. (Large LLMs like Phi-3.5 Mini would use ~3.5 GB RAM ², so a rule-based approach is much lighter.)
- **Injection:** The polished text is automatically injected at the current cursor using the clipboard and simulated keyboard. The program copies the text to the clipboard and triggers a “Ctrl+V” (or Cmd+V on Mac) to paste it ³ ⁴. This makes the text appear instantly in **any** app (WhatsApp, Slack, browser, etc.) without manual typing.

2. Technology Stack

- **Hotkey listener:** `pynput` (global keyboard hooks) ³. Allows defining a global `<modifier>+<key>` combination that starts/stops recording.
- **Audio capture:** `sounddevice` library. Opens a stream and buffers audio while the hotkey is held. (On key release, the stream stops.)
- **Voice Activity Detection (VAD):** `Silero VAD` for detecting speech. Faster-Whisper can use Silero under the hood (via `vad_filter=True` in `model.transcribe`) ¹. Silero VAD is extremely fast and small (~2 MB) ⁵ – processing 30 ms of audio in <1 ms on CPU ⁶ – and filters out silence to save CPU.
- **Speech-to-Text:** `Faster-Whisper` (a CPU-optimized Whisper implementation). Use a small quantized model, e.g. `"tiny.en"` or `"base"` with `compute_type="int8"` for minimal memory. E.g.:

```
from faster_whisper import WhisperModel
model = WhisperModel("tiny.en", device="cpu", compute_type="int8")
segments, _ = model.transcribe("audio.wav")
text = "".join(segment.text for segment in segments)
```

(Faster-Whisper's examples ⁷ show how to instantiate on CPU with `int8` quantization, which runs very fast on 8 GB machines.)

- **Formatting:** Simple Python rules or a tiny LLM:
- **Rule-based:** Detect bullets ("first, second" → `•` list; punctuation fixes; emoji keywords → `👉` etc). This uses negligible memory.
- **LLM option (optional):** If an LLM is desired, options include GPT4All or Llama variants. For example, GPT4All can load models via `llama.cpp` (e.g. Mistral-7B in 4-bit ~3–4 GB ⁸). **Caution:** even quantized LLMs (e.g. Meta-Llama-3-8B Q4_0) are multi-GB ⁸. We generally recommend rule-based to stay ultra-light.
- **Clipboard & Paste:** Use `pyperclip` to copy text and `pynput.keyboard.Controller` to simulate keys. Example paste:

```
import pyperclip
from pynput.keyboard import Key, Controller
pyperclip.copy(final_text)
k = Controller()
# Paste (Windows)
k.press(Key.ctrl); k.press('v')
k.release('v'); k.release(Key.ctrl)
```

Pynput's docs ³ ⁴ show how to press/release keys programmatically.

3. Hotkey and Recording Flow

1. **User presses the hotkey.** A global keyboard listener (via `pynput`) catches `<Win>+<Fn>` or chosen combo ³ and starts the recording process.
2. **Start recording audio.** Create a `sounddevice.InputStream` and begin buffering input frames. In parallel, pass audio through VAD. Only keep frames classified as speech.
3. **User releases the hotkey.** Stop the audio stream. The buffered audio (e.g. written to an in-memory WAV or NumPy array) is ready for transcription.
4. **Transcription & VAD filtering.** Optionally call `model.transcribe(..., vad_filter=True)` ¹ to automatically cut silence. Get a text string from the segments.
5. **Text formatting.** Run formatting rules on the raw text: add capitalization/punctuation, emoji substitutions, bullet lists, etc.
6. **Auto-paste.** Copy the polished text to clipboard and simulate `Ctrl+V`. The text appears at the cursor location in whatever app is active.

This sequence ensures near-instant input. Real-world testing shows the **total latency ~1–2 seconds** (mostly transcription time). On a modern 8 GB laptop, tiny-int8 Whisper runs in under 1 s, and text processing is instantaneous.

4. Performance and Requirements

- **Memory:** Minimal. Faster-Whisper tiny-int8 ~250 MB RAM; Silero VAD ~5 MB; plus Python overhead. **Total RAM usage ~300 MB** while dictating, and ~<100 MB idle.
- **Storage:** Whisper model files (tiny+ base: ~500 MB total), silence model (2 MB), app code. ~1 GB disk.

- **CPU:** Runs on CPU (no GPU needed). VAD saves CPU by ignoring silence.
- **Latency:** ~0.5–1s transcription for a few seconds of speech on a typical i5 laptop.

By contrast, a local LLM (like Phi-3.5 Mini) would need ~3.5 GB RAM ². Using such a heavy model would blow our 8 GB target. That's why this design **avoids large LLMs**. Instead, we rely on lightweight models and deterministic rules for formatting.

5. Android Port (Phase 2)

To bring this to Android, we use native libraries (since Python stacks are heavy on mobile):

- **Speech Recognition:** Use [whisper.cpp](#) (C++ port of Whisper). Many developers have built Android apps with whisper.cpp; for example, the *WhisperCppAndroidDemo* shows tiny and base models performing well ⁹.
- **Implementation:** In an Android app (Kotlin/Java), call the whisper.cpp library via JNI. Load the tiny or base model (around 10–50 MB compressed). Record mic audio (Android AudioRecord) and feed it to whisper.cpp for transcription.
- **Formatting:** Similar rule-based post-processing in Java/Kotlin (or use a small on-device model if absolutely needed).
- **Injection:** On Android, injecting text into any field typically requires an **accessibility service** (or using `InputConnection` for a custom keyboard). Alternatively, one can copy text to the clipboard and prompt the user (long-press “Paste”), or implement a floating overlay that pastes. These are engineering details beyond this spec, but in principle the same clipboard+paste idea can be adapted to Android APIs.
- **Privacy:** All models run on-device. No data leaves the phone.

6. Key References and Tools

- **Faster-Whisper:** Python library to run Whisper models on CPU. Install with `pip install faster-whisper`. Usage example from docs ⁷:

```
from faster_whisper import WhisperModel
model = WhisperModel("tiny", device="cpu", compute_type="int8")
segments, info = model.transcribe("audio.wav")
```

(Supports `vad_filter=True` to drop silence ¹.)

- **Silero VAD:** Voice activity detection. Lightweight (~2 MB) and very fast ⁶ ⁵. Use it to segment out speech.
- **pynput:** Allows global hotkey listening and keyboard control. See examples of `HotKey` for combo detection ³ and `Controller` for sending keys ⁴.
- **pyperclip:** Cross-platform clipboard utility (copy text to clipboard).
- **LLM Libraries (optional):** GPT4All with `llama.cpp` bindings can run models like Mistral-7B-Q4_0 (~3–4 GB) ⁸. Ollama's Phi-3.5 Mini is ~2.2 GB on disk and requires ~3.5 GB RAM ² – too large for this use. We therefore default to **no LLM or a very small one**.
- **Android whisper.cpp demo:** [WhisperCppAndroidDemo by Digipom](#) – shows Whisper running on Android.

7. Development Deliverables

The completed project should deliver:

1. **Windows Application:** A background Python app or executable that listens for the hotkey and performs speech-to-text as described.
2. **Hotkey Configuration:** Allow user to customize the shortcut key.
3. **Installation Script:** Instructions or script to install required Python packages and download models (e.g. `faster-whisper` and model weights).
4. **Documentation:** A user guide on usage (how to choose microphone, set hotkey, etc.) and on resource usage (approx RAM).
5. **Android Prototype (Phase 2):** An Android app or service demonstrating the same behavior (hotkey replaced by a floating button or long-press) using `whisper.cpp`.

8. Conclusion

This design achieves an **always-on, privacy-first voice typing assistant** with minimal resources. It replicates Wispr Flow's core features **offline** and **cost-free**. By carefully choosing lightweight, quantized models and rule-based processing, the tool uses only a few hundred megabytes of RAM (instead of several gigabytes). All components are open-source and free, making the total cost ₹0.

Ready to build! Provide this specification to the developer/team to implement the FlowType assistant.

1 7 `faster-whisper` · PyPI
<https://pypi.org/project/faster-whisper/>

2 Phi-3.5 Mini: Best AI for 4GB RAM Laptops (Beats Llama 3.2 3B) | Local AI Master
<https://localaimaster.com/models/phi-3-5-mini>

3 4 Handling the keyboard — pynput 1.7.6 documentation
<https://pynput.readthedocs.io/en/latest/keyboard.html>

5 6 GitHub - snakers4/silero-vad: Silero VAD: pre-trained enterprise-grade Voice Activity Detector · GitHub
<https://github.com/snakers4/silero-vad>

8 GitHub - nomic-ai/gpt4all: GPT4All: Run Local LLMs on Any Device. Open-source and available for commercial use. · GitHub
<https://github.com/nomic-ai/gpt4all>

9 Android example app using `whisper.cpp` from ggerganov · openai whisper · Discussion #691 · GitHub
<https://github.com/openai/whisper/discussions/691>